

Large Language Model-Based Agents for Software Engineering: A Survey

Modern software development is transitioning from usage of basic AI tools that simply generate output and is moving towards LLM-based agents. These agents are built to interact with the real world, which means they can run code, check for errors, and use feedback to improve their work. This ability to work in loops and learn from the environment is what makes them so much more effective than traditional models for complex software tasks. To function effectively, these agents rely on a core framework that mimics human problem-solving. The framework consists of four core components, such as, planning, memory, perception, and action. Planning enables agents to break big goals into smaller steps, while a memory system keeps track of historical actions and feedback. Perception refers to the processing of inputs such as documentation, or logs, and action enables agents to interact with the environment through external tools. This setup ensures the agent isn't just guessing but is verifying its work against real-world constraints.

LLM-based agents are being applied across a wide range of activities in the framework. In requirements engineering, agents simulate multiple stakeholders to elicit, negotiate, and refine user requirements, helping uncover ambiguities and conflicting needs. Agents move beyond one-shot synthesis in code generation by incorporating iterative feedback loops. They even execute generated code to obtain compiler or runtime feedback and can consult humans to clarify intent. When it comes to software testing and debugging, agents generate test cases and refine, or patch based on fault detection feedback. These workflows resemble human debugging practices and demonstrate how agents can assist in complex maintenance tasks.

The survey highlights the rise of multi-agent systems, where different AI models take on specialized roles like a Product Manager, a Software Architect, or a Tester. By having these agents collaborate and critique each other's work, researchers can simulate the workflow of an entire software team. Agents employ diverse planning representations, memory designing to support long horizon tasks, and action modules rely heavily on tool usage. The survey also emphasizes the importance of human agent collaboration at critical stages. This hybrid approach helps align agent behavior with human intent and increases trust in agent generated outputs.

A notable limitation of the survey is that it does not explicitly analyze the criticality, or replaceability of individual agent roles within multi-agent software engineering workflows. As a result, it remains unclear how robust these systems are to partial agent failure or misbehavior, such as hallucination (1) by a single agent. If a key agent, say the one responsible for planning, begins hallucinating, the error may propagate through the workflow, leading to incorrect output. An explicit role based ablation study would increase the reliability of these agent-based SE systems in real-world deployments.

As an application, I will apply the end-to-end, role based multi-agent framework (2) described in the paper to my research on defending LLMs against adversarial attacks. Specialized agents can be assigned roles such as malicious input detection, intent verification, token evaluation, and output validation to detect the adversarial input. The defense strategy will aim to identify adversaries that attempt to manipulate model outputs and enhance the overall robustness of LLM-based systems.

(1) – Section 4.2, Code Generation, Large Language Model-Based Agents for Software Engineering: A Survey
(2) – Section 4.7.2, Role Specialization of Software Development Team, Large Language Model-Based Agents for Software Engineering: A Survey